

NLP 202: Phrase-Structure Parsing

Jeffrey Flanigan

Winter 2023

University of California Santa Cruz

jmflanig@ucsc.edu

- Push-Down Automata (PDAs)
- Treebanks
- Shift-Reduce Parsing
- Evaluation
- CKY Parsing

Context-free grammar

N	Finite set of non-terminal symbols	NP, VP, S
Σ	Finite alphabet of terminal symbols	the, dog, a
R	Set of production rules, each $A \rightarrow \beta$ $\beta \in (\Sigma, N)$	$S \rightarrow NP VP$ Noun $\rightarrow$ dog
S	Start symbol	

FSAs are the machines that recognize regular languages.

What machine recognizes context-free languages?

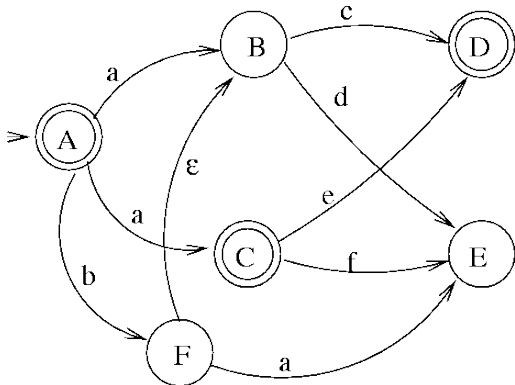
FSAs are the machines that recognize regular languages.

What machine recognizes context-free languages?

Push-Down Automata (PDAs)

Review from last month

- FSAs: finite state automata
 - A finite-state automaton is a device that can be in one of a finite number of states



Pushdown Automata (PDA)

- PDAs: a more powerful computation device that can recognize CFLs.
- PDA: a ϵ -NFA with a “**stack**” which serves as “memory”, and store a string of stack symbols.
- LR $\leftrightarrow$ FSA
- CFL $\leftrightarrow$ PDA
- The general non-deterministic version accepts ALL CFLs.
- The deterministic version accepts only a subsets of the CFLs.

Formal definition of a PDA

- $M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$

Q, Σ, q_0, F are the same as in FSAs. Which is?

Γ : set of stack symbols (finite)
(denoted using X, Y, Z)

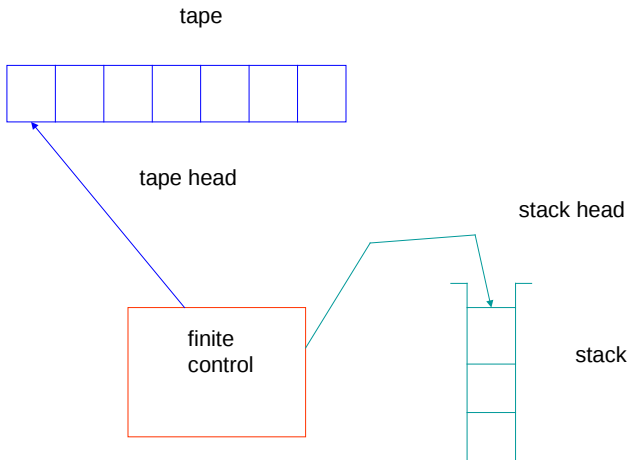
z_0 : start stack symbol.

$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ transitions

Formal definition of a PDA

- Q : a finite set of states (like in finite automaton)
- Σ : a finite set of input symbols (like in finite automaton)
- Γ : a finite stack alphabet. The components that we are allowed to push onto the stack.
- δ : the transition function $\delta(q,a,X)$ where:
 - q is a state in Q .
 - a is either an input symbol in Σ or $a = \epsilon$
 - $X \in \Gamma$ (X is a stack symbol)
- q_0 : the start state.
- Z_0 : the start symbol,
- F : the set of accepting states.

Pushdown Automata (PDA)

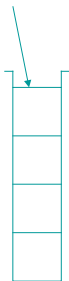


Pushdown Automata (PDA)

p	u	s	h	d	o	w	n
---	---	---	---	---	---	---	---

The **tape** is divided into finitely many cells.
Each cell contains a symbol in an alphabet Σ .

Pushdown Automata (PDA)



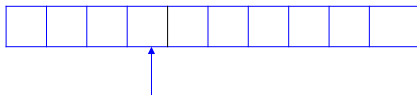
The **stack** head always scans the top symbol of the stack. It performs two basic operations:

Push: **add** a new symbol at the top.

Pop: **read** and **remove** the top symbol.

Alphabet of stack symbols: **Γ**

Pushdown Automata (PDA)



- The **head/pointer** scans at a cell on the **tape** and can *read* a symbol on the cell. In each move, the head can move to the right cell.

Pushdown Automaton (PDA)

- Transitions depends on
 - **Current state.**
 - **Input.**
(these two, the same as FSAs)
 - **Top of the stack.**
(which is the new thing)

Pushdown Automaton (PDA)

- After each transition:
 - **New state.**
 - **Pop the stack : ϵ**
 - **And push the stack with new symbol.**

Pushdown Automaton (PDA)

- In one transition the PDA may do the following:
 - Consume the input symbol. If ϵ is the input symbol, then no input is consumed.
 - Go to a new state, which may be the same as the previous state.
 - Replace the symbol at the top of the stack by any string.
 - If this string is ϵ then this is a **pop** of the stack
 - The string might be the same as the current stack top (does **nothing**)
 - Replace with a new string (**pop** and **push**)
 - Replace with multiple symbols (multiple **pushes**)

Example

$$L = \{0^n 1^n \mid n \geq 0\}$$

Is there a PDA that recognizes this?

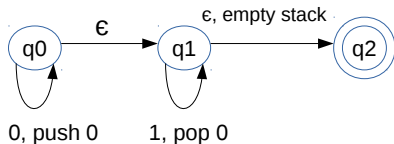
PDA. Simple example

$$L = \{0^n 1^n \mid n \geq 0\}.$$

- A PDA is able to recognize this language!
 - Can use its stack to store the number of 0's it has seen.
 - As each 0 is read, **push** it onto the stack
 - As soon as 1's are read, **pop** a 0 off the stack
 - If reading the input is finished exactly when the stack is empty, **accept** the input else **reject** the input

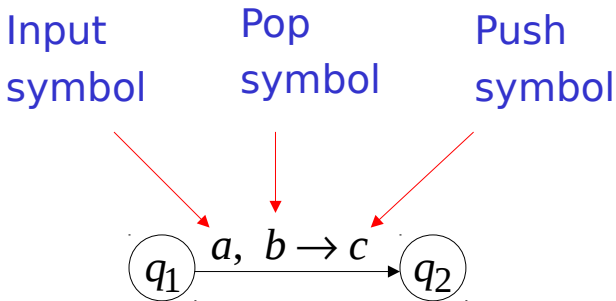
PDA. Simple example

$$L = \{0^n 1^n \mid n \geq 0\}.$$



- q_0 : push 0.
- q_1 : pop all 0s while reading 1s.
- q_2 : accept when we have an empty stack.

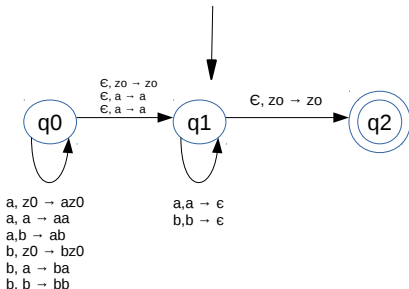
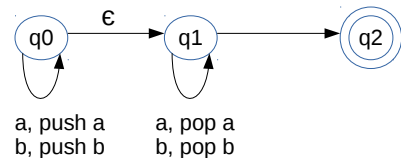
Pushdown Automata (PDA)



Transition

Pushdown Automata (PDA)

$$L = \{ w wr \mid w \in \{a,b\}^* \}$$



More about PDAs

- CFGs and PDAs are equivalent: For every CFG, you can construct a PDA that recognizes the CFG's language, and vice-versa
- Not all PDAs can be determinized
- Languages representable by deterministic PDAs are subset of context-free languages

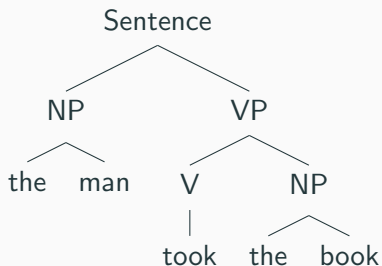
LR Parsers and GLR Parsers

- For a given a CFG, you can construct a deterministic PDA that attempts to parse inputs, but will fail to correctly recognize some inputs
 - Example: LR(0) or Simple LR(0) parsers
- LR parsers will work correctly for special classes of CFGs
- **Generalized LR (GLR)** parsers, which incorporate a backtracking mechanism, extend LR parsers to handle arbitrary CFGs

We will learn about other parsing techniques today and this week

- Building a CFG by hand is really hard
- Rather than create the rules by hand, we can annotate sentences with their syntactic structure and then extract the rules from the annotations
- **Treebanks**: collections of sentences annotated with syntactic structure

The First Phrase-Structure Tree (?)



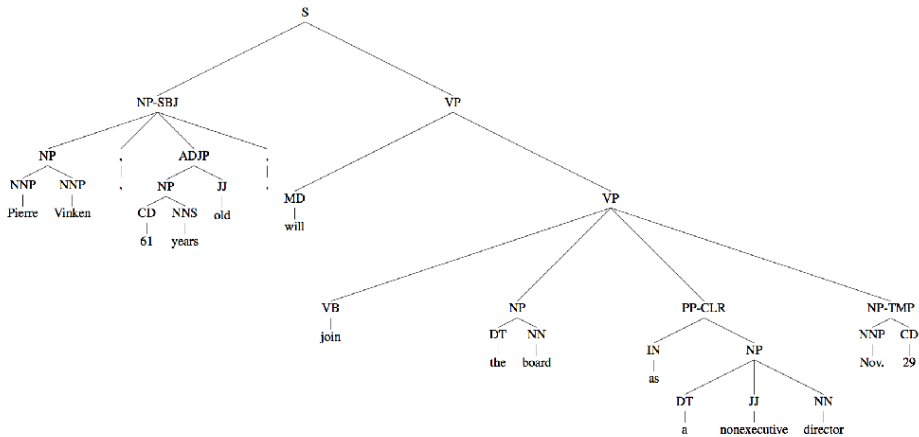
The Penn Treebank

- The first large-scale treebank created was the Penn Treebank (?)
- 3 million words
- 8 years to develop (1989-1996)
- [Annotation guidelines](#)

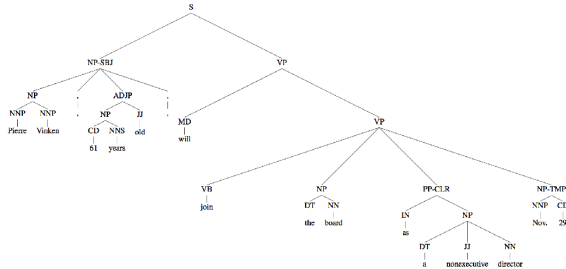
Notable Annotation Decisions

- Flat noun phrases - **doesn't analyse noun phrases too heavily, since can get complicated**
- Includes extra information like traces and semantic information on the nodes (often removed, parsers usually don't produce these)

Penn Treebank



Penn Treebank



NP	→	NNP NNP
NP-SBJ	→	NP , ADJP ,
S	→	NP-SBJ VP
VP	→	VB NP PP-CLR NP-TMP

Example rules extracted from this single annotation

Penn Treebank

NP → DT JJ NN
NP → DT JJ NNS
NP → DT JJ NN NN
NP → DT JJ JJ NN
NP → DT JJ CD NNS
NP → RB DT JJ NN NN
NP → RB DT JJ JJ NNS
NP → DT JJ JJ NNP NNS
NP → DT NNP NNP NNP NNP JJ NN
NP → DT JJ NNP CC JJ JJ NN NNS
NP → RB DT JJS NN NN SBAR
NP → DT VBG JJ NNP NNP CC NNP
NP → DT JJ NNS , NNS CC NN NNS NN
NP → DT JJ JJ VBG NN NNP NNP FW NNP
NP → NP JJ , JJ ‘ ‘ SBAR ‘ ’ NNS

LISP Encoding in the Penn Treebank

```
( (S
  (NP-SBJ-1
    (NP (NNP Rudolph) (NNP Agnew) )
    ( , , )
    (UCP
      (ADJP
        (NP (CD 55) (NNS years) )
        (JJ old) )
      (CC and)
      (NP
        (NP (JJ former) (NN chairman) )
        (PP (IN of)
          (NP (NNP Consolidated) (NNP Gold) (NNP Fields) (NNP PLC) ))))
      ( , , ) )
    (VP (VBD was)
      (VP (VBN named)
        (S
          (NP-SBJ (-NONE- *-1) )
          (NP-PRD
            (NP (DT a) (JJ nonexecutive) (NN director) )
            (PP (IN of)
              (NP (DT this) (JJ British) (JJ industrial) (NN conglomerate) )))))
          ( . . ) ))
```

With an annotated treebank, you can build parsers using machine learning

- [Online demo](#) (Stanford parser)
- Modern parsers don't reject sentences, they always attempt to give a parse
- They may or may not use CFGs to produce the parses

Modern parsers treat the problem as a machine learning problem: learn to produce y from x from a dataset of (x, y) pairs, where x is a sentence and y is a parse tree.

Our first parsing algorithm: Shift-reduce parsing

Assume we are given a CFG (created by hand, extracted from a treebank, etc)

Shift-Reduce Parsing: Overview

- Shift-reduce transition based process
- A **Stack**: holds the partial parse trees already built
- A **Buffer (or queue)**: holds the incoming words (and/or parts-of-speech)
- **Actions**: SHIFT, REDUCE

Overview

- The parsing process starts with all words in the buffer.
- In each step, there is a deterministic decision that decides which action to take given the parsing state.
- We use a classifier that decides what to do given the parsing state.
 - Parsing state: Stack and Buffer.

Shift-Reduce Parsing: Example

$S \rightarrow NP VP$

$S \rightarrow VP$

$NP \rightarrow DT NN$

$VP \rightarrow VB$

$VP \rightarrow VB NN$

$DT \rightarrow \text{that}|\text{this}|\text{a}|\text{the}$

$NN \rightarrow \text{book}|\text{flight}|\text{meal}$

$VB \rightarrow \text{book}|\text{include}|\text{read}$

Shift-Reduce Parsing: Example

Book that flight.

Stack



Buffer

Book that flight

Shift-Reduce Parsing: Example

Book that flight.

Stack



Buffer

Book that flight

SHIFT

Shift-Reduce Parsing: Example

Book that flight.

Stack

Book

Buffer

that flight

Shift-Reduce Parsing: Example

Book that flight.

Stack

Book

Buffer

that flight

REDUCE 1 VB (using grammar rule $VB \rightarrow \text{book}$)

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB

Buffer

that flight

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB

Buffer

that flight

SHIFT

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB that

Buffer

flight

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB that

Buffer

flight

REDUCE 1 DT (using grammar rule $DT \rightarrow \text{that}$)

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB DT

Buffer

flight

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB DT

Buffer

flight

SHIFT

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB DT flight

Buffer

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB DT flight

Buffer

REDUCE 1 NN (using grammar rule $NN \rightarrow \text{flight}$)

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB DT NN

Buffer

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB DT NN

Buffer

REDUCE 2 NP (using grammar rule $NP \rightarrow DT\ NN$)

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB NP

Buffer

Shift-Reduce Parsing: Example

Book that flight.

Stack

VB NP

Buffer

REDUCE 2 VP (using grammar rule $VP \rightarrow VB\ NP$)

Shift-Reduce Parsing: Example

Book that flight.

Stack

VP

Buffer

Shift-Reduce Parsing: Example

Book that flight.

Stack

VP

Buffer

REDUCE 1 S (using grammar rule $S \rightarrow VP$)

Shift-Reduce Parsing: Example

Book that flight.

Stack

S

Buffer

Shift-Reduce Parsing: Example

Book that flight.

Stack

S

Buffer

Have empty buffer, and S in the stack. Parse successful!

To recover the parse tree, we can keep track of the partially built tree structure in the symbols in the stack.

Shift-Reduce Parsing: Example

Book that flight.

Stack



Buffer

Book that flight

Shift-Reduce Parsing: Example

Book that flight.

Stack



Buffer

Book that flight

SHIFT

Shift-Reduce Parsing: Example

Book that flight.

Stack

Book

Buffer

that flight

Shift-Reduce Parsing: Example

Book that flight.

Stack

Book

Buffer

that flight

REDUCE 1 VB

Shift-Reduce Parsing: Example

Book that flight.

Stack

(VB Book)

Buffer

that flight

Shift-Reduce Parsing: Example

Book that flight.

Stack

(VB Book) that

Buffer

flight

SHIFT

Shift-reduce parsing

Similar to a PDA, but with a key difference:

- Deterministic decision that decides which action to take: in NLP, usually a classifier. **Most recently, this classifier is a neural network, see for example ?**

Running time:

- Linear in the length of the sentence $O(n)$
- In terms of memory, it is also cheap

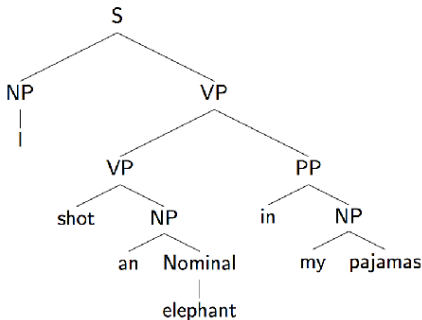
Evaluation

Parseval (1991):

Represent each tree as a collection of tuples:

$\langle l_1, i_1, j_1 \rangle, \dots, \langle l_n, i_n, j_n \rangle$

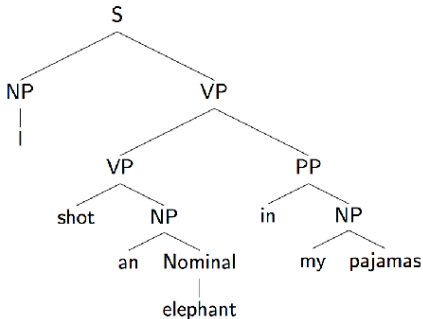
- l_k = label for kth phrase
- i_k = index for first word in kth phrase
- j_k = index for last word in kth phrase

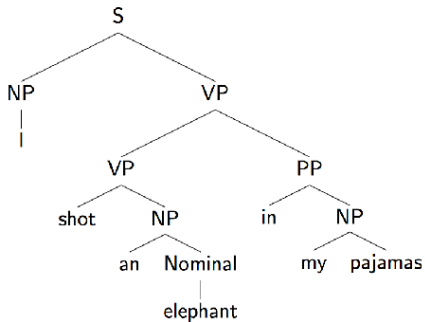
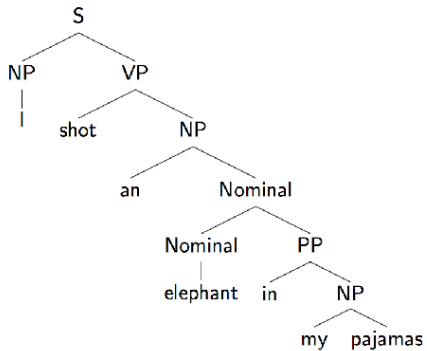


Evaluation

I₁ shot₂ an₃ elephant₄ in₅ my₆ pajamas₇

- <S, 1, 7>
- <NP, 1, 1>
- <VP, 2, 7>
- <VP, 2, 4>
- <NP, 3, 4>
- <Nominal, 4, 4>
- <PP, 5, 7>
- <NP, 6, 7>





Evaluation

I₁ shot₂ an₃ elephant₄ in₅ my₆ pajamas₇

- <S, 1, 7>
- <NP, 1, 1>
- <VP, 2, 7>
- <VP, 2, 4>
- <NP, 3, 4>
- <Nominal, 4, 4>
- <PP, 5, 7>
- <NP, 6, 7>

- <S, 1, 7>
- <NP, 1, 1>
- <VP, 2, 7>
- <NP, 3, 7>
- <Nominal, 4, 7>
- <Nominal, 4, 4>
- <PP, 5, 7>
- <NP, 6, 7>

Evaluation

Calculate precision, recall, F1 from these collections of tuples

- Precision: number of tuples in tree 1 also in tree 2, divided by number of tuples in tree 1
- Recall: number of tuples in tree 1 also in tree 2, divided by number of tuples in tree 2

Evaluation

I₁ shot₂ an₃ elephant₄ in₅ my₆ pajamas₇

- <S, 1, 7>
- <NP, 1, 1>
- <VP, 2, 7>
- <VP, 2, 4>
- <NP, 3, 4>
- <Nominal, 4, 4>
- <PP, 5, 7>
- <NP, 6, 7>

- <S, 1, 7>
- <NP, 1, 1>
- <VP, 2, 7>
- <NP, 3, 7>
- <Nominal, 4, 7>
- <Nominal, 4, 4>
- <PP, 5, 7>
- <NP, 6, 7>

CKY Parsing



CKY algorithm (aka CYK)

- Cocke-Kasami-Younger algorithm
 - Independently discovered in late 60s / early 70s
- An efficient bottom up parsing algorithm for (P)CFGs
 - can be used both for the recognition and parsing problems
 - Very important in NLP (and beyond)
- We will start with the non-probabilistic version

CKY

- Cocke-Kasami-Younger algorithm (also CYK) for parsing from a grammar expressed in CNF.
 - Kasami (1965)
 - Younger (1967)
 - Cocke and Schwartz (1970)
- Bottom-up dynamic programming: once we discover a constituent, we can make it available for any rule that needs it.



Constraints on the grammar

- The basic CKY algorithm supports only rules in the **Chomsky Normal Form (CNF)**:

$$C \rightarrow x$$

Unary **preterminal** rules (generation of words given PoS tags)

$$N \rightarrow \text{telescope} \quad D \rightarrow \text{the}$$

$$C \rightarrow C_1 C_2$$

Binary **inner** rules $S \rightarrow NP VP$ $NP \rightarrow D N$



Constraints on the grammar

- The basic CKY algorithm supports only rules in the **Chomsky Normal Form (CNF)**:

$$C \rightarrow x$$

$$C \rightarrow C_1 C_2$$

- Any CFG can be converted to an equivalent CNF
 - Equivalent means that they define **the same language**
 - However (syntactic) **trees will look differently**
 - It is possible to address it by defining such transformations that allows for easy **reverse transformation**



Transformation to CNF form

- What one need to do to convert to CNF form
 - Get rid of rules that mix terminals and non-terminals
 - Get rid of unary rules: $C \rightarrow C_1$
 - Get rid of N-ary rules: $C \rightarrow C_1 C_2 \dots C_n$ ($n > 2$)

Crucial to process them, as required for efficient parsing



Transformation to CNF form: binarization

- Consider $NP \rightarrow DT\ NNP\ VBG\ NN$



- How do we get a set of binary rules which are equivalent?



Transformation to CNF form: binarization

- Consider $NP \rightarrow DT \ NNP \ VBG \ NN$



- How do we get a set of binary rules which are equivalent?

$NP \rightarrow DT \ X$

$X \rightarrow NNP \ Y$

$Y \rightarrow VBG \ NN$



Transformation to CNF form: binarization

- Consider $NP \rightarrow DT \ NNP \ VBG \ NN$



- How do we get a set of binary rules which are equivalent?

$NP \rightarrow DT \ X$

$X \rightarrow NNP \ Y$

$Y \rightarrow VBG \ NN$

- A more systematic way to refer to new non-terminals

$NP \rightarrow DT \ @NP|DT$

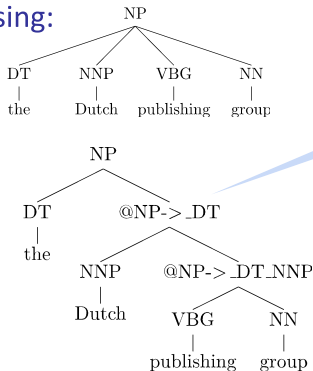
$@NP|DT \rightarrow NNP \ @NP|DT_NNP$

$@NP|DT_NNP \rightarrow VBG \ NN$



Transformation to CNF form: binarization

- Instead of binarizing tuples we can binarize trees on preprocessing:



Also known as **lossless Markovization** in the context of PCFGs

Can be easily reversed on postprocessing

S	→	NP VP
VP	→	VBD NP
VP	→	VP PP
Nominal	→	Nominal PP
Nominal	→	pajamas elephant I
PP	→	IN NP
NP	→	DT NN
NP	→	pajamas elephant I
NP	→	PRP\$ Nominal

VBD	→	shot
DT	→	an my
PRP	→	I
PRP\$	→	my
IN	→	in

I shot an elephant in my pajamas

0 I 1 shot 2 an 3 elephant 4 in 5 my 6 pajamas 7

Bottom-up dynamic programming: **once we discover a constituent, we can make it available for any rule that needs it.**

We're going to use a **chart** to keep track of constituents we can build.

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]						
	VBD [1,2]					
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Each cell i,j keeps track of all phrase types that can be formed from *all* words from position i through position j

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]						
	VBD [1,2]					
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

What phrases can be formed
from "shot an elephant in"

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]						
	VBD [1,2]					
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

What phrases can be formed
from "I shot an elephant in my
pajamas"

CNF

- In CNF, each non-terminal generates two non-terminals

$S \rightarrow NP VP$

[S [NP I] [VP shot an elephant in my pajamas]]

- If the left-side non-terminal (S) spans tokens $i-j$, the right side (NP VP) must also span $i-j$, and there must be a single position k that separates them.

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]						
	VBD [1,2]					
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Does any rule generate PRP
VBD?

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅					
	VBD [1,2]					
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Does any rule generate
VBD DT?

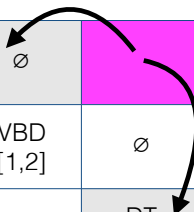
I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅					
	VBD [1,2]	∅				
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Two possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

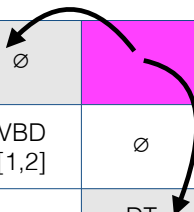
NP, PRP [0,1]	∅					
	VBD [1,2]	∅				
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]



Two possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅					
	VBD [1,2]	∅				
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]



Two possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅				
	VBD [1,2]	∅				
		DT [2,3]				
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

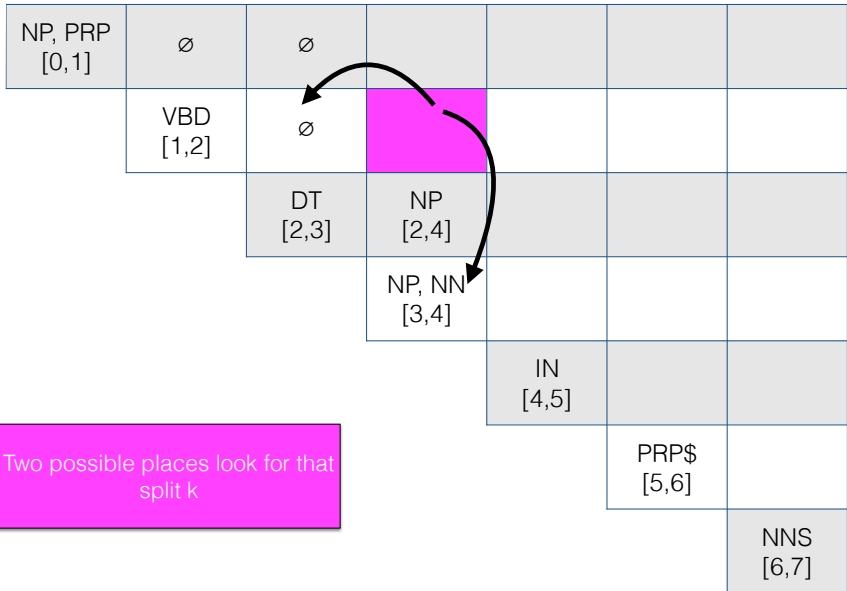
Does any rule generate
DT NN?

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅				
	VBD [1,2]	∅				
		DT [2,3]	NP [2,4]			
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Two possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------



I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅				
	VBD [1,2]	∅	VP [1,4]			
		DT [2,3]	NP [2,4]			
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Three possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅				
	VBD [1,2]	∅	VP [1,4]			
		DT [2,3]	NP [2,4]			
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Three possible places look for that split k

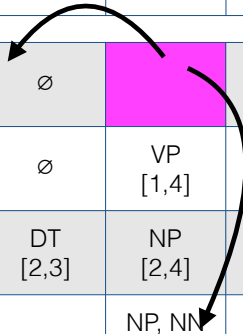
I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅				
	VBD [1,2]	∅	VP [1,4]			
		DT [2,3]	NP [2,4]			
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

Three possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅				
	VBD [1,2]	∅	VP [1,4]			
		DT [2,3]	NP [2,4]			
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]



Three possible places look for that split k

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]			
	VBD [1,2]	∅	VP [1,4]			
		DT [2,3]	NP [2,4]			
			NP, NN [3,4]			
				IN [4,5]		
					PRP\$ [5,6]	
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	
			NP, NN [3,4]	∅	∅	
				IN [4,5]	∅	
					PRP\$ [5,6]	
						NNS [6,7]

*elephant in

*an elephant in

*shot an elephant in

*I shot an elephant in

*in my

*elephant in my

*an elephant in my

*shot an elephant in my

*I shot an elephant in my

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	
			NP, NN [3,4]	∅	∅	
				IN [4,5]	∅	
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	
			NP, NN [3,4]	∅	∅	
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	
		DT [2,3]	NP [2,4]	∅	∅	NP [3,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

The diagram illustrates the syntactic tree structure for the sentence "I shot an elephant in my pajamas". The tree is represented by a grid of cells, each containing a phrase label and its span. The cells are arranged in a triangular pattern, with the root node 'S' at the top. The phrase labels and their spans are: NP, PRP [0,1], VBD [1,2], DT [2,3], NP [2,4], NP, NN [3,4], IN [4,5], PRP\$ [5,6], NP [5,7], and NNS [6,7]. The empty cells (∅) represent the words of the sentence. Arrows indicate the phrase structure relationships: a magenta arrow from the root 'S' to 'NP, PRP [0,1]', a black arrow from 'S' to 'VP [1,4]', a magenta arrow from 'VP [1,4]' to 'VBD [1,2]', a black arrow from 'VP [1,4]' to 'NP [2,4]', a magenta arrow from 'NP [2,4]' to 'DT [2,3]', a black arrow from 'NP [2,4]' to 'NP, NN [3,4]', a magenta arrow from 'NP, NN [3,4]' to 'NP [3,7]', a black arrow from 'NP, NN [3,4]' to 'IN [4,5]', a magenta arrow from 'IN [4,5]' to 'PP [4,7]', a black arrow from 'PP [4,7]' to 'PRP\$ [5,6]', and a magenta arrow from 'PP [4,7]' to 'NP [5,7]'. The magenta arrows represent the phrase structure tree, while the black arrows represent the constituent structure tree.

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	VP ₁ , VP ₂ [1,7]
		DT [2,3]	NP [2,4]	∅	∅	NP [2,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	
	VBD [1,2]	∅	VP [1,4]	∅	∅	VP ₁ , VP ₂ [1,7]
		DT [2,3]	NP [2,4]	∅	∅	NP [2,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

Possibilities:

$S_1 \rightarrow NP VP_1$

$S_2 \rightarrow NP VP_2$

$? \rightarrow S PP$

$? \rightarrow PRP VP_1$

$? \rightarrow PRP VP_2$

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	S ₁ , S ₂ [0,7]
	VBD [1,2]	∅	VP [1,4]	∅	∅	VP ₁ , VP ₂ [1,7]
		DT [2,3]	NP [2,4]	∅	∅	NP [2,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

Success! We've recognized a total of two valid parses

CKY algorithm

```
function CKY-PARSE(words, grammar) returns table

  for  $j \leftarrow$  from 1 to LENGTH(words) do
    for all  $\{A \mid A \rightarrow \text{words}[j] \in \text{grammar}\}$ 
       $\text{table}[j-1, j] \leftarrow \text{table}[j-1, j] \cup A$ 
    for  $i \leftarrow$  from  $j-2$  downto 0 do
      for  $k \leftarrow i+1$  to  $j-1$  do
        for all  $\{A \mid A \rightarrow BC \in \text{grammar} \text{ and } B \in \text{table}[i, k] \text{ and } C \in \text{table}[k, j]\}$ 
           $\text{table}[i, j] \leftarrow \text{table}[i, j] \cup A$ 
```

Figure 12.5 The CKY algorithm.

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

NP, PRP [0,1]	∅	∅	S [0,4]	∅	∅	S ₁ , S ₂ [0,7]
	VBD [1,2]	∅	VP [1,4]	∅	∅	VP ₁ , VP ₂ [1,7]
		DT [2,3]	NP [2,4]	∅	∅	NP [2,7]
			NP, NN [3,4]	∅	∅	NP [3,7]
				IN [4,5]	∅	PP [4,7]
					PRP\$ [5,6]	NP [5,7]
						NNS [6,7]

Runtime complexity?

PCFG

- Probabilistic context-free grammar: each production is also associated with a probability.
- This lets us calculate the probability of a parse for a given sentence; for a given parse tree T for sentence S comprised of n rules from R (each $A \rightarrow \beta$):

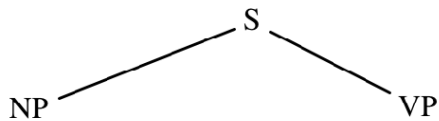
$$P(T, S) = \prod_i^n P(\beta \mid A)$$

PCFG

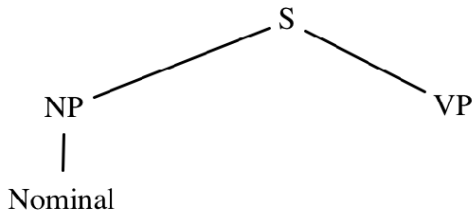
N	Finite set of non-terminal symbols	NP, VP, S
Σ	Finite alphabet of terminal symbols	the, dog, a
R	Set of production rules, each $A \rightarrow \beta$ [p] $p = P(\beta \mid A)$	$S \rightarrow NP VP$ Noun $\rightarrow$ dog
S	Start symbol	

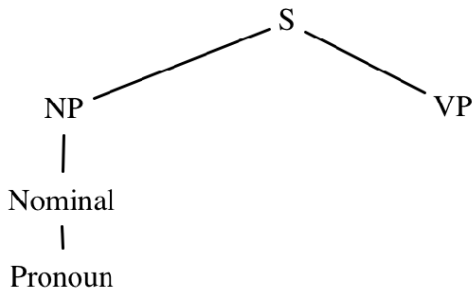
S

$$P(\text{NP VP} \mid \text{S})$$

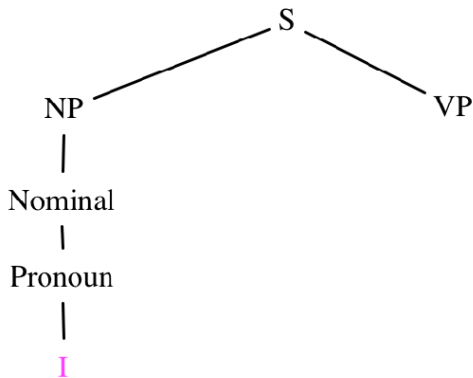


$$P(\text{NP VP} \mid \text{S}) \\ \times P(\text{Nominal} \mid \text{NP})$$

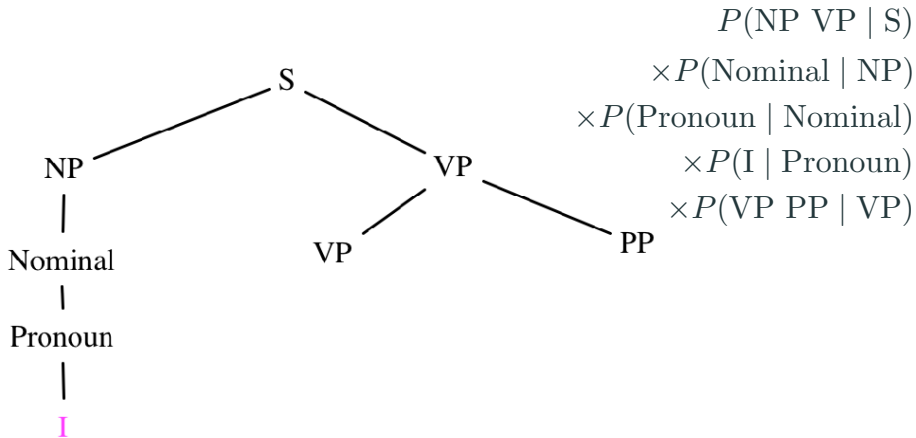


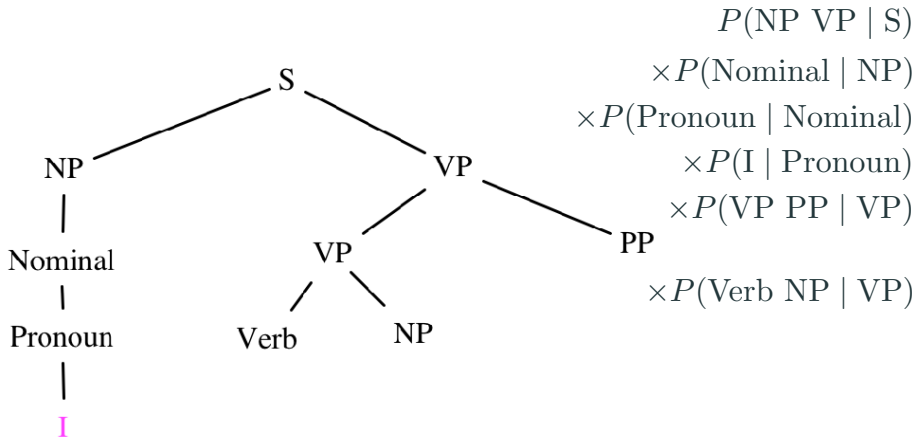


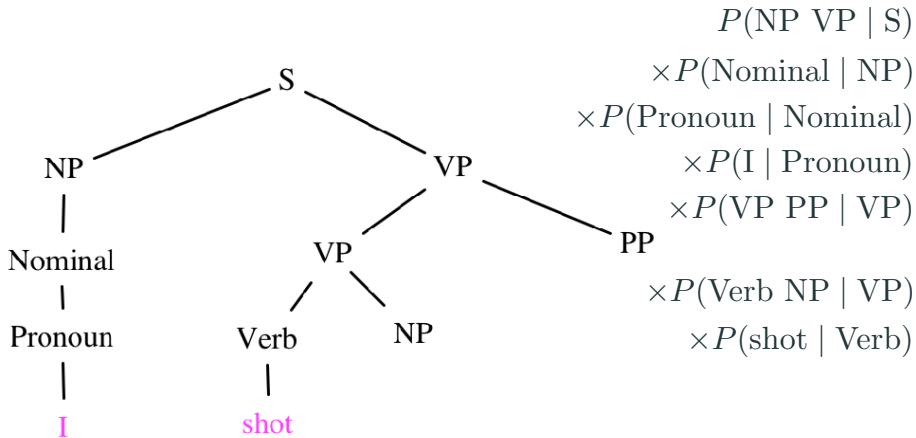
$$\begin{aligned} &P(\text{NP VP} \mid \text{S}) \\ &\times P(\text{Nominal} \mid \text{NP}) \\ &\times P(\text{Pronoun} \mid \text{Nominal}) \end{aligned}$$

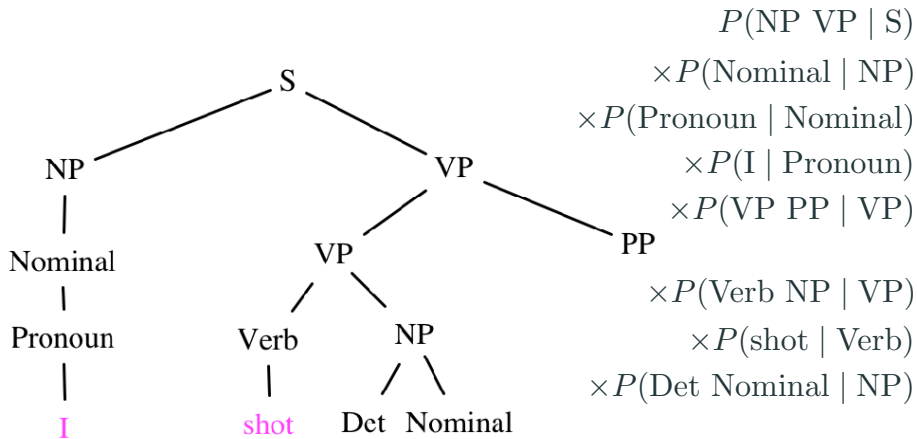


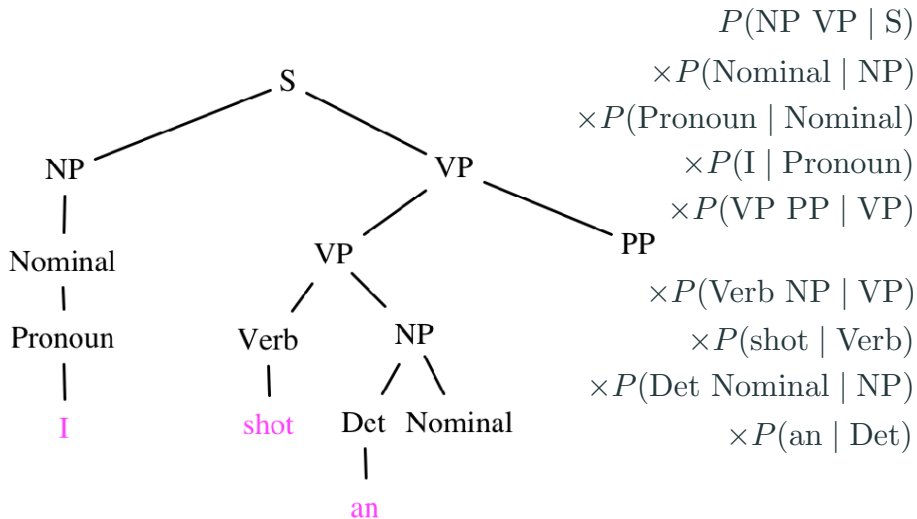
$$\begin{aligned} &P(\text{NP VP} \mid \text{S}) \\ &\times P(\text{Nominal} \mid \text{NP}) \\ &\times P(\text{Pronoun} \mid \text{Nominal}) \\ &\times P(\text{I} \mid \text{Pronoun}) \end{aligned}$$

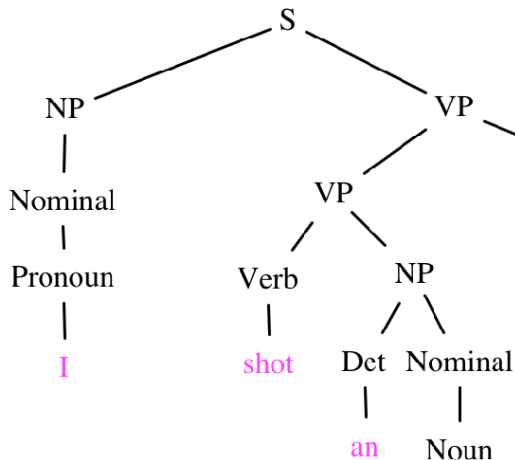




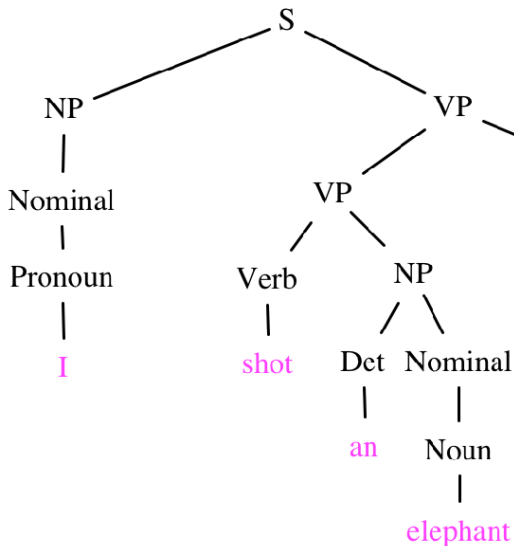




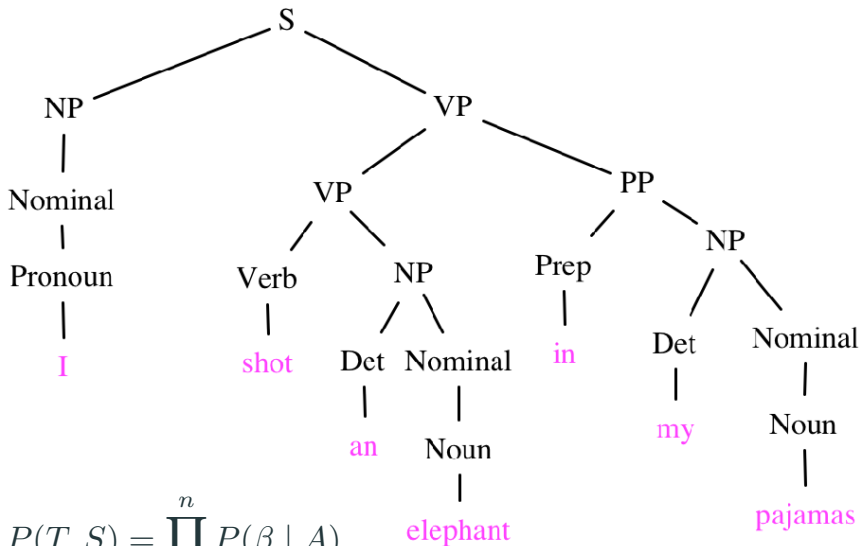




$$\begin{aligned}
 &P(\text{NP VP} \mid \text{S}) \\
 &\times P(\text{Nominal} \mid \text{NP}) \\
 &\times P(\text{Pronoun} \mid \text{Nominal}) \\
 &\times P(\text{I} \mid \text{Pronoun}) \\
 &\times P(\text{VP PP} \mid \text{VP}) \\
 &\times P(\text{Verb NP} \mid \text{VP}) \\
 &\times P(\text{shot} \mid \text{Verb}) \\
 &\times P(\text{Det Nominal} \mid \text{NP}) \\
 &\times P(\text{an} \mid \text{Det}) \\
 &\times P(\text{Noun} \mid \text{Nominal})
 \end{aligned}$$



$$\begin{aligned} &P(\text{NP VP} \mid \text{S}) \\ &\times P(\text{Nominal} \mid \text{NP}) \\ &\times P(\text{Pronoun} \mid \text{Nominal}) \\ &\times P(\text{I} \mid \text{Pronoun}) \\ &\times P(\text{VP PP} \mid \text{VP}) \\ &\times P(\text{Verb NP} \mid \text{VP}) \\ &\times P(\text{shot} \mid \text{Verb}) \\ &\times P(\text{Det Nominal} \mid \text{NP}) \\ &\times P(\text{an} \mid \text{Det}) \\ &\times P(\text{Noun} \mid \text{Nominal}) \\ &\times P(\text{elephant} \mid \text{Noun}) \end{aligned}$$



$$P(T, S) = \prod_i^n P(\beta \mid A)$$

Weighted CFGs

- In general, can have a weighted CFG with arbitrary weights
- The score for a tree is the semiring product $\otimes$ of the rule weights

Finding the highest scoring parse tree

Keep track of the highest score for each non-terminal built:

$$table(i, j, A) = \max_k P(A \rightarrow BC) \times table(i, k, B) \times table(k, j, C)$$

CFG

- This use of CKY allows us to:
 - check whether a sentence is grammatical in the language defined by the CFG
 - enumerate all possible parses for a sentence
- But it doesn't tell us on its own which of those possible parses is most likely.

PCFGs

- A PCFG gives us a mechanism for assigning scores (here, probabilities) to different parses for the same sentence.
- But we often care about finding the single best parse with the highest probability.
- We calculate the max probability parse using CKY by storing the probability of each phrase within each cell as we build it up.

Finding the highest scoring parse tree

Keep track of the highest score for each non-terminal built:

$$table(i, j, A) = \max_k P(A \rightarrow BC) \times table(i, k, B) \times table(k, j, C)$$

We can use CKY to do many things

Find the highest scoring parse tree (**Viterbi decoding**):

$$table(i, j, A) = \max_k P(A \rightarrow BC) \times table(i, k, B) \times table(k, j, C)$$

Sum over all parse trees (**marginalize over the parse trees**):

$$table(i, j, A) = \sum_k P(A \rightarrow BC) \times table(i, k, B) \times table(k, j, C)$$

Semiring version:

$$table(i, j, A) = \bigoplus_k P(A \rightarrow BC) \otimes table(i, k, B) \otimes table(k, j, C)$$

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

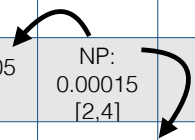
PRP:0.04 [0,1]						
	VBD:0.04 [1,2]					
		DT:0.05 [2,3]				
			NN:0.03 [3,4]			
				IN:0.10 [4,5]		
					PRP\$: 0.12 [5,6]	
						NNS:0.01 [6,7]

Probability of a terminal (word)
given its tag

$$P(A \rightarrow \beta)$$

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

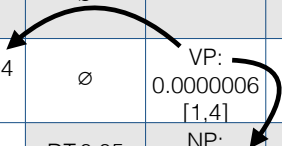
PRP:0.04 [0,1]	∅	∅				
	VBD:0.04 [1,2]	∅				
		DT:0.05 [2,3]	NP: 0.00015 [2,4]			
			NN:0.03 [3,4]			
				IN:0.10 [4,5]		
					PRP\$:0.12 [5,6]	
						NNS:0.01 [6,7]



$table(2, 4, NP) = P(NP \rightarrow DT \ NN) \times table(2, 3, DT) \times table(3, 4, NN)$

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

PRP:0.04 [0,1]	∅	∅				
	VBD:0.04 [1,2]	∅	VP: 0.0000006 [1,4]			
		DT:0.05 [2,3]	NP: 0.00015 [2,4]			
			NN:0.03 [3,4]			
				IN:0.10 [4,5]		
					PRP\$:0.12 [5,6]	
						NNS:0.01 [6,7]

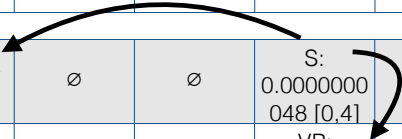


We just calculated this value
and can use it now

$$table(1,4,VP) = P(VP \rightarrow VBD NP) \times table(1,2,VBD) \times table(2,4,NP)$$

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

PRP:0.04 [0,1]	∅	∅	S: 0.0000000 048 [0,4]			
	VBD:0.04 [1,2]	∅	VP: 0.0000006 [1,4]			
		DT:0.05 [2,3]	NP: 0.00015 [2,4]			
			NN:0.03 [3,4]			
				IN:0.10 [4,5]		
					PRP\$:0.12 [5,6]	
						NNS:0.01 [6,7]



We just calculated this value
and can use it now

$$table(0, 4, S) = P(S \rightarrow NP VP) \times table(0, 1, NP) \times \text{table}(1, 4, VP)$$

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

PRP:0.04 [0,1]	∅	∅	S: 0.0000000 048 [0,4]			
	VBD:0.04 [1,2]	∅	VP: 0.0000006 [1,4]			
		DT:0.05 [2,3]	NP: 0.00015 [2,4]			
			NN:0.03 [3,4]			
				IN:0.10 [4,5]		
					PRP\$:0.12 [5,6]	
						NNS:0.01 [6,7]

Note these values are getting very small! Better to add in log space

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

PRP: -3.21 [0,1]	∅	∅	S: -19.2 [0,4]			
	VBD: -3.21 [1,2]	∅	VP: -14.3 [1,4]			
		DT: -3.0 [2,3]	NP: -8.8 [2,4]			
			NN: -3.5 [3,4]			
				IN: -2.3 [4,5]		
					PRP\$: -2.12 [5,6]	
						NNS: -4.6 [6,7]

Note these values are getting very small! Better to add in log space

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

PRP: -3.21 [0,1]	∅	∅	S: -19.2 [0,4]	∅	∅	
	VBD: -3.21 [1,2]	∅	VP: -14.3 [1,4]	∅	∅	VP ₁ , VP ₂ [1,7]
		DT: -3.0 [2,3]	NP: -8.8 [2,4]	∅	∅	NP: -24.7 [2,7]
			NN: -3.5 [3,4]	∅	∅	NP: -19.4 [3,7]
				IN: -2.3 [4,5]	∅	PP: -13.0 [4,7]
					PRP\$: -2.12 [5,6]	NP: -9.0 [5,7]
						NNS: -4.6 [6,7]

For any phrase type spanning [i,j], we only need to keep the max probability given the assumptions of a PCFG

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

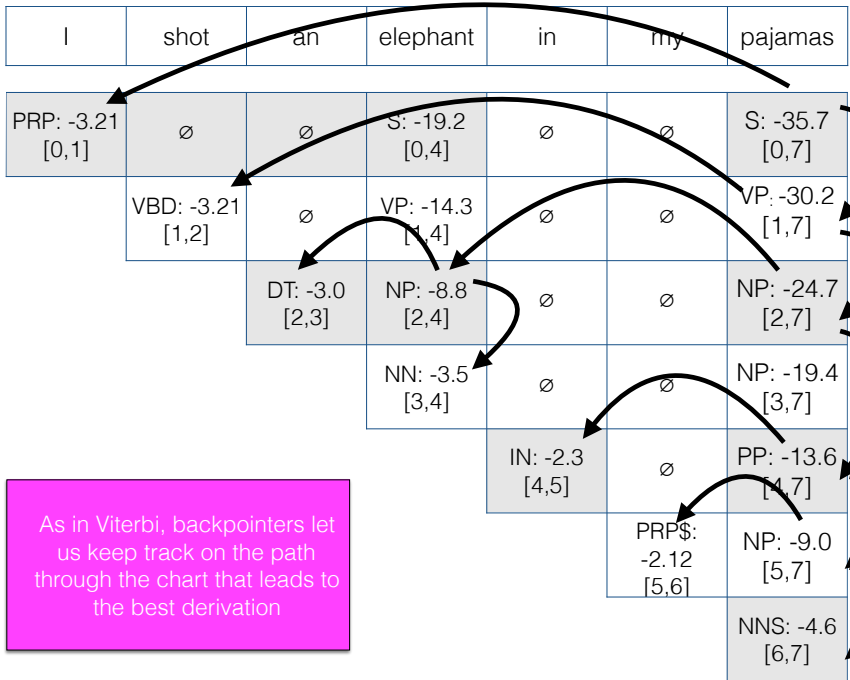
PRP: -3.21 [0,1]	∅	∅	S: -19.2 [0,4]	∅	∅	
	VBD: -3.21 [1,2]	∅	VP: -14.3 [1,4]	∅	∅	VP: -30.2 [1,7]
		DT: -3.0 [2,3]	NP: -8.8 [2,4]	∅	∅	NP: -24.7 [2,7]
			NN: -3.5 [3,4]	∅	∅	NP: -19.4 [3,7]
				IN: -2.3 [4,5]	∅	PP: -13.6 [4,7]
					PRP\$: -2.12 [5,6]	NP: -9.0 [5,7]
						NNS: -4.6 [6,7]

For any phrase type spanning [i,j], we only need to keep the max probability given the assumptions of a PCFG

I	shot	an	elephant	in	my	pajamas
---	------	----	----------	----	----	---------

PRP: -3.21 [0,1]	∅	∅	S: -19.2 [0,4]	∅	∅	S: -35.7 [0,7]
	VBD: -3.21 [1,2]	∅	VP: -14.3 [1,4]	∅	∅	VP: -30.2 [1,7]
		DT: -3.0 [2,3]	NP: -8.8 [2,4]	∅	∅	NP: -24.7 [2,7]
			NN: -3.5 [3,4]	∅	∅	NP: -19.4 [3,7]
				IN: -2.3 [4,5]	∅	PP: -13.6 [4,7]
					PRP\$: -2.12 [5,6]	NP: -9.0 [5,7]
						NNS: -4.6 [6,7]

For any phrase type spanning [i,j], we only need to keep the max probability given the assumptions of a PCFG



Algorithms that parse without converting to CNF (binarizing) first:

- CKY+ (??)
- Earley parsing (?)

Semiring parsing:

- Introduced by ?

Synchronous Context-Free Grammars (SCFGs)

The “transducer” version of CFGs.

Used extensively in statistical machine translation.

Like a CFG, but each rule generates two sides, separated by :

Japanese CFG:

$$S \rightarrow NP VP$$

$$VP \rightarrow NP V$$

$$V \rightarrow tabeta$$

$$NP \rightarrow jon - ga$$

$$NP \rightarrow ringo - o$$

Synchronous Context-Free Grammars (SCFGs)

The “transducer” version of CFGs.

Used extensively in statistical machine translation.

Like a CFG, but each rule generates two sides, separated by :

Japanese-English SCFG:

$$\begin{aligned} S &\rightarrow NP[1] VP[2] : NP[1] VP[2] \\ VP &\rightarrow NP[1] V[2] : V[2] NP[1] \\ V &\rightarrow tabeta : ate \\ NP &\rightarrow jon - ga : John \\ NP &\rightarrow ringo - o : an apple \end{aligned}$$

Indices on non-terminals keep track of reorderings.

Synchronous generation

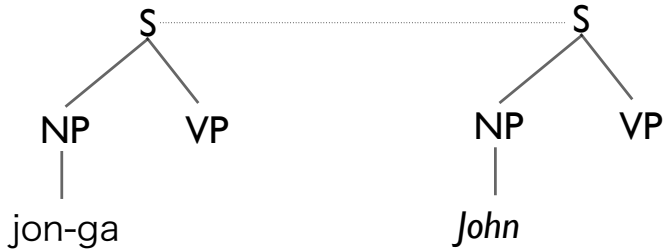
Synchronous generation

S S

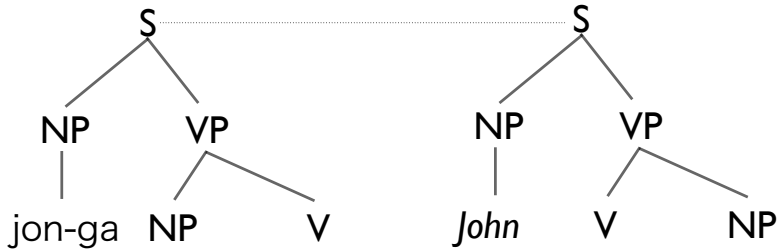
Synchronous generation



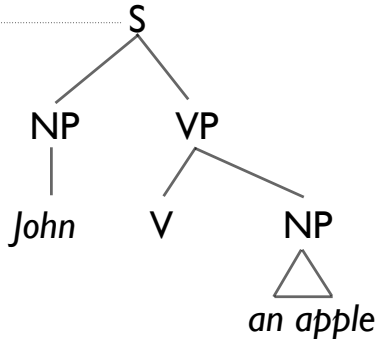
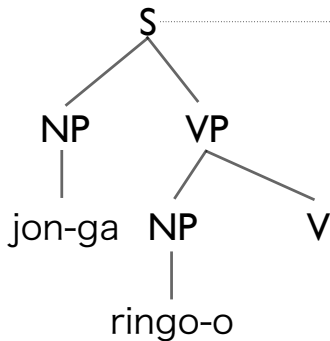
Synchronous generation



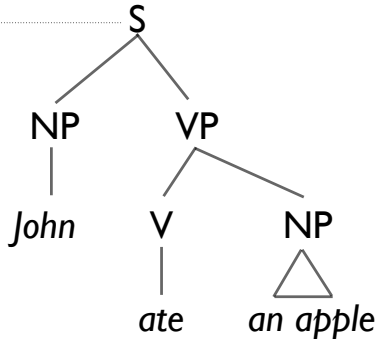
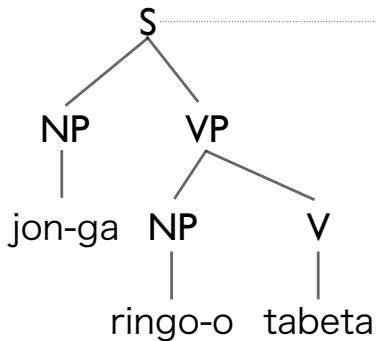
Synchronous generation



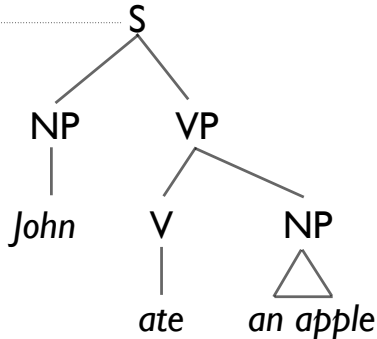
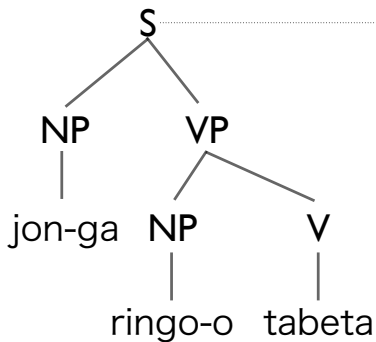
Synchronous generation



Synchronous generation



Synchronous generation



Output: (jon-ga ringo-o tabeta : *John ate an apple*)

